

Day 4 Assignment

Async IO in Python: Fetching Several APIs at Once

John Michael Bilbao
Techstart
August 11, 2026

Introduction

This assignment asks for a Python program that fetches data from several APIs at the same time using `asyncio` and `aiohttp`, and a short report on why doing it that way helps.

I wrote three small scripts. `async_fetch.py` is the assignment itself: it fetches five endpoints concurrently and prints what came back. `compare.py` runs the same requests one after another and then all at once, so the benefit is measured rather than asserted. `errors.py` shows what happens when some of the endpoints fail, which turned out to be the part I learned the most from.

All of them use `jsonplaceholder.typicode.com`, a free JSON API that needs no key.

Step 1: Setting up the environment

`aiohttp` is the only thing that needs installing. `asyncio` is in the standard library.

```
$ pip install aiohttp
Successfully installed aiohttp-3.13.5

import asyncio
import aiohttp
```

`asyncio` provides the event loop and the tools for running coroutines together. `aiohttp` is an HTTP client built to work with it. The ordinary `requests` library cannot be used here: its calls block the thread, so the event loop would sit still during every request and nothing would overlap.

Steps 2 and 3: The `async` function and the main function

`fetch_data` sends one GET request and returns the decoded JSON. The important line is the `await`: while the network is busy, `await` gives control back to the event loop, which is free to start or continue the other requests. Without it there would be no concurrency, only a more complicated way of writing a loop.

`main` builds one coroutine per URL and hands them all to `asyncio.gather`, which runs them together and returns the results in the order the URLs were listed, not the order the responses arrived.

```
• async_fetch.py  compare.py  errors.py

23 async def fetch_data(session: aiohttp.ClientSession, url: str) -> dict:
24     """Send a GET request and return the decoded JSON body.
25
26     await hands control back to the event loop while the network is busy, which
27     is what lets the other requests run during this one's waiting time.
28     """
29     started = time.perf_counter()
30     async with session.get(url, timeout=aiohttp.ClientTimeout(total=10)) as response:
31         response.raise_for_status()
32         data = await response.json()
33     elapsed = time.perf_counter() - started
34     print(f" {response.status} {elapsed:5.2f}s {url}")
35     return data
36
37
38 async def main() -> None:
39     print(f"Fetching {len(URLS)} endpoints concurrently\n")
40     started = time.perf_counter()
41
42     async with aiohttp.ClientSession() as session:
43         # One coroutine per URL, then gather runs them together and gives the
44         # results back in the order the URLs were listed.
45         tasks = [fetch_data(session, url) for url in URLS]
46         results = await asyncio.gather(*tasks)
47
48     total = time.perf_counter() - started
49
50     print(f"\nGot {len(results)} responses in {total:.2f}s\n")
51     for url, data in zip(URLS, results):
52         label = url.rsplit("/", 2)[-2]
53         title = data.get("title") or data.get("name") or data.get("body", "")
54         print(f" {label:9} id={data.get('id')} {title[:46].strip()}")

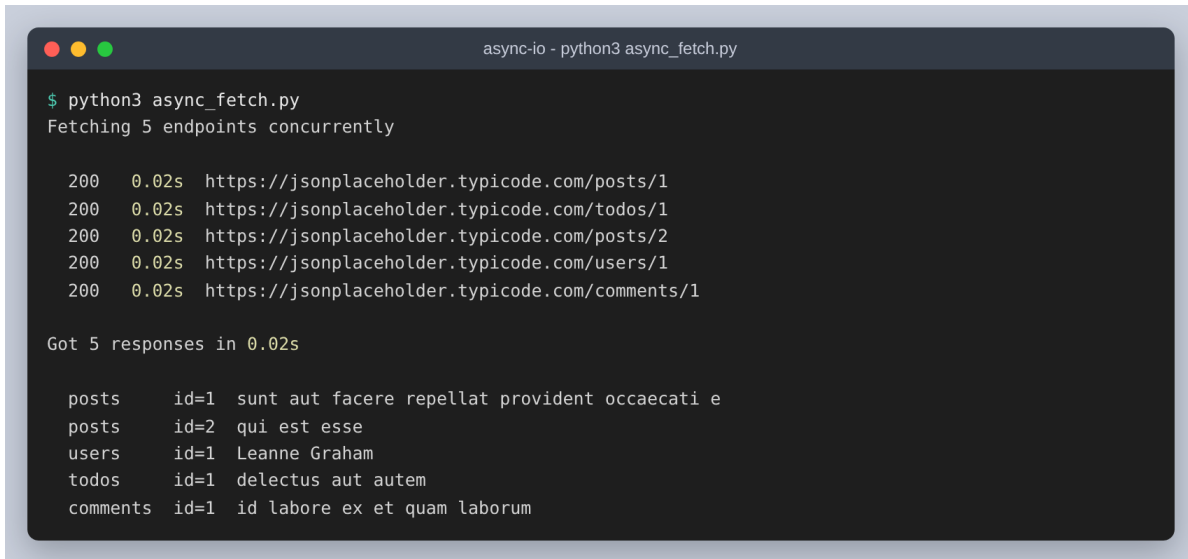
async_fetch.py Python Spaces: 4 UTF-8
```

Figure 1: `fetch_data` and `main` from `async_fetch.py`

Two details worth pointing out. The session is created once and passed in, because a `ClientSession` holds a connection pool and making a new one per request throws that away. And nothing is awaited inside the loop that builds the list: awaiting there would finish each request before starting the next, which is the usual way of accidentally writing sequential code that looks asynchronous.

Step 4: Running the program

The five requests all finish in about the time one of them takes, and the responses come back out of order while the printed results stay in the order the URLs were given.



```
async-io - python3 async_fetch.py

$ python3 async_fetch.py
Fetching 5 endpoints concurrently

200 0.02s https://jsonplaceholder.typicode.com/posts/1
200 0.02s https://jsonplaceholder.typicode.com/todos/1
200 0.02s https://jsonplaceholder.typicode.com/posts/2
200 0.02s https://jsonplaceholder.typicode.com/users/1
200 0.02s https://jsonplaceholder.typicode.com/comments/1

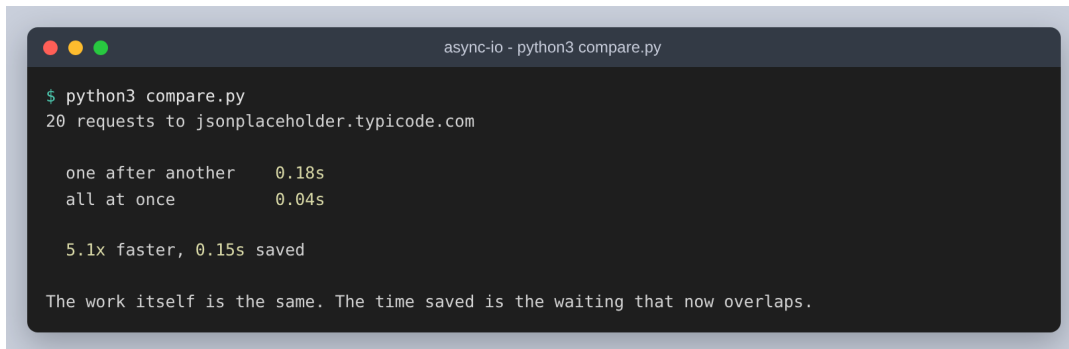
Got 5 responses in 0.02s

posts    id=1  sunt aut facere repellat provident occaecati e
posts    id=2  qui est esse
users    id=1  Leanne Graham
todos    id=1  delectus aut autem
comments id=1  id labore ex et quam laborum
```

Figure 2: The output of python3 async_fetch.py

Step 5: Why this is worth doing

Five requests against this API are too quick to compare meaningfully, so compare.py uses 20 and warms the connection up first, which keeps DNS and the TLS handshake out of the measurement.



```
async-io - python3 compare.py

$ python3 compare.py
20 requests to jsonplaceholder.typicode.com

one after another    0.18s
all at once          0.04s

5.1x faster, 0.15s saved

The work itself is the same. The time saved is the waiting that now overlaps.
```

Figure 3: The same requests done sequentially and then concurrently

One after another takes 0.18 seconds. All at once takes 0.04, which is 5.1 times faster and saves 0.15 seconds.

The reason is that almost none of that time is Python doing work. It is time spent waiting for the network, and waiting is something a program can do for twenty connections as easily as for one. Sequentially the waits happen one after the other and add up; concurrently they overlap, so the total is close to the slowest single request instead of the sum of all of them.

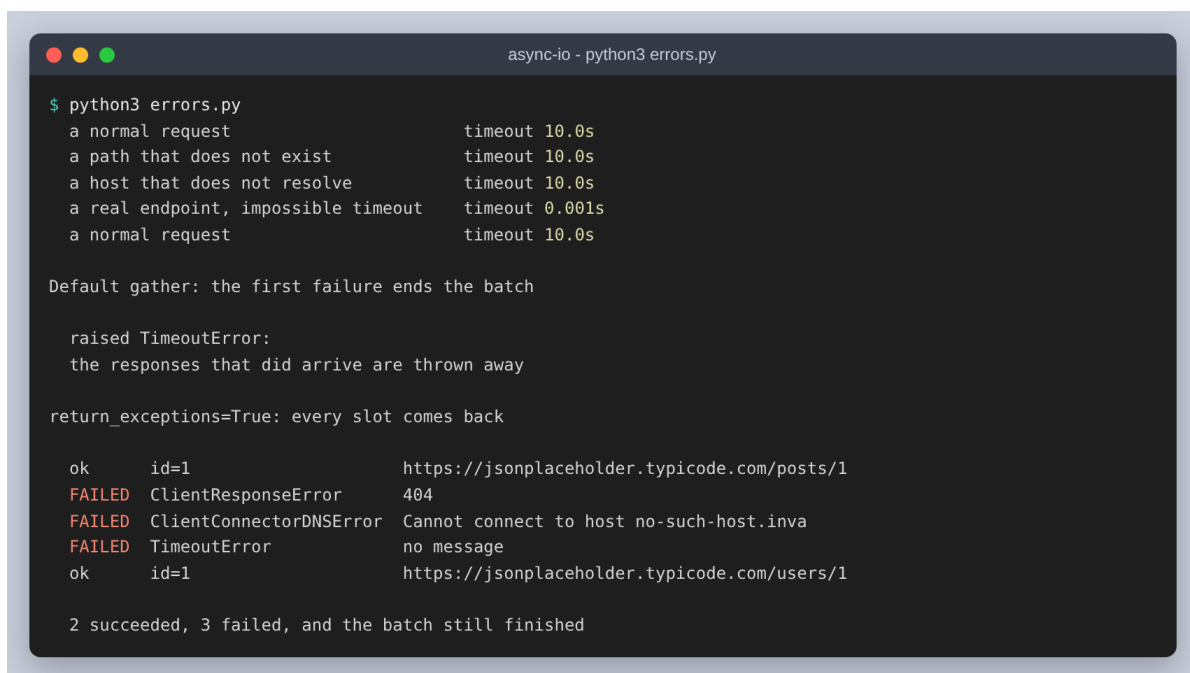
This is also why async is the right tool for this problem and the wrong tool for many others. It only helps when the program is waiting on something external. Work that keeps the CPU busy gets no benefit at all, because there is no idle time to reuse, and a long calculation inside a coroutine will block the event loop and stall every other task with it.

Threads could also overlap the waiting, but each thread needs its own stack and the operating system has to switch between them. Here 20 concurrent requests run on one thread, and the only places control changes hands are the awaits, which are visible in the source.

Errors, and what gather does with them

This is where my first version was wrong. `asyncio.gather` stops at the first exception, and the results that had already arrived are lost with it. One bad URL out of five and the whole batch returns nothing.

`errors.py` runs a deliberately broken list: a working endpoint, a path that returns 404, a hostname that does not resolve, and a real endpoint given a timeout of a thousandth of a second so it cannot possibly finish.



```
async-io - python3 errors.py

$ python3 errors.py
a normal request                timeout 10.0s
a path that does not exist       timeout 10.0s
a host that does not resolve     timeout 10.0s
a real endpoint, impossible timeout 0.001s
a normal request                timeout 10.0s

Default gather: the first failure ends the batch

raised TimeoutError:
the responses that did arrive are thrown away

return_exceptions=True: every slot comes back

ok      id=1                https://jsonplaceholder.typicode.com/posts/1
FAILED  ClientResponseError  404
FAILED  ClientConnectorDNSError Cannot connect to host no-such-host.inva
FAILED  TimeoutError        no message
ok      id=1                https://jsonplaceholder.typicode.com/users/1

2 succeeded, 3 failed, and the batch still finished
```

Figure 4: The same batch with and without `return_exceptions=True`

With the default, the run ends with a `TimeoutError` and the two responses that succeeded are thrown away. Passing `return_exceptions=True` changes the failures into ordinary return values: each slot in the result list holds either the data or the exception object, so the two that worked still come back and the three that failed can be reported. The three failures arrive as different types, which is useful, because a 404 is a problem with the request and a DNS error or a timeout is a problem with the connection.

The other thing this made clear is that a timeout has to be set. Without one, a single endpoint that never answers keeps the whole gather waiting indefinitely, and concurrency does not save you from that.

What I took away from it

The syntax is the easy part. `async def`, `await` and `asyncio.gather` took a few minutes to get working, and the program was fetching five endpoints at once almost immediately.

What actually needed thinking about was everything around it: that awaiting inside the loop would have quietly made the whole thing sequential again, that the session has to be shared for the connection pool to be any use, that `gather` throws away good results when one task fails unless you ask it not to, and that without a timeout the batch can hang forever on one bad endpoint. Those are the parts that would have caused problems in something real, and none of them are visible from the syntax alone.